

CS F342 COMPUTER ARCHITECTURE PROJECT REPORT

TOPIC: 5 STAGE PIPELINE WITH HAZARD DETECTION AND DATA FORWARDING FEATURES

LAB SECTION : P1
GROUP NUMBER: 7
PROBLEM SET : 7

MEMBERS:

HANUSH NAG ABHIRAM M	- 2022A3PS0549H
VARUN PRAKASH	- 2022A8PS1542H
DEBANGSHU PATNAIK	- 2022A3PS1468H
HARSHIT VERMA	- 2022A3PS0510H

CONTENTS:

- 1) Problem statement
- 2) Given information and expected outputs
- 3) Full code for the processor
- 4) Architecture of the processor
- 5) Individual block description
- 6) Testbench and simulation
- 7) Timeline of execution
- 8) Cycle by cycle explanation
- 9) Extra

PROBLEM STATEMENT

Project Set - 7

Statement: Design a 5-stage (as discussed in the lecture) pipeline RISC processor (with hazard detection and data forwarding unit as necessary) that can execute the following instructions:

0000 SUB reg1, reg2, reg3
0004 NAND reg4, reg1, reg5
0008 LW reg7, 3(reg6)
0012 AND reg8, reg7, reg9
0016 ADDI reg10, reg8, 1010

Initialize the register file with the following data

reg2 = 82344 reg3 = 20413
reg5 = 16ACD reg6 = 9
reg9 = A123E

The instruction format is as follows:

31	28	27	24	23	20	19	16	15	0
OPCODE	Dest Reg (WN)			Source Reg 1 (RN1)		Source Reg 2 (RN2)		Immediate value	

Opcode for each instruction:

Instruction 1: 0000
Instruction 2: 0001
Instruction 3: 0011
Instruction 4: 0111
Instruction 5: 1111

The processor has the following control signals:

- ALUSrc - Select the second input of ALU
- ALUOp (2 bits) - Control ALU operation
- MR - Read data from memory
- MW - Write data into memory
- MReg - Move data from memory to register
- EnIM - Read instruction memory contents
- EnRW - Write data into the register file
- FA - Forward A mux control (used in data forwarding circuitry)
- FB - Forward B mux control (used in data forwarding circuitry)
- IFIDWrite - Disable IF/ID change (used in hazard detection circuit)

Initialize PC with all zeros. The instruction memory is of size 32 bytes, and the processor has 16 registers, numbered from reg0 to reg15, and the registers are 32 bits wide. A read operation from the instruction memory outputs 4 consecutive bytes of information (starting from the byte address provided to the memory) at the positive edge of the clock if the EnIM control signal is high. Assume that the register file has two 32-bit read ports: RD1 and RD2, for data reading and one 32-bit write port: WD, for data writing. At the rising edge of the clock, the read ports RD1 and RD2 output the data from the registers whose addresses are available at RN1 and RN2, respectively. At the falling edge of a clock, data is written via the write port WD to the register file whose address is present at WN if the EnRW signal is true. Design the data memory size as per the requirement. All the data are in hexadecimal format unless specified.

Draw the detailed architecture-level diagram of the processor, depicting all the blocks (e.g., register file, instruction memory, ALU, PC, combinational functional blocks, hazard detection unit, forwarding unit, etc.). Describe each block individually, and create behavioral verilog models for each architectural block separately. Build the top-level structural model of the processor by instantiating and interconnecting the individual architectural blocks. Specify the size and format of all pipeline registers including the fields holding the decoded control signals as well as data. Show all the input, output, and control signal waveforms in the report.

INSTRUCTIONS TO BE EXECUTED

0000 SUB reg1, reg2, reg3
0004 NAND reg4, reg1, reg5
0008 LW reg7, 3(reg6)
0012 AND reg8, reg7, reg9
0016 ADDI reg10, reg8, 1010

INITIAL VALUES

reg2 = 82344
reg3 = 20413
reg5 = 16ACD
reg6 = 9
reg9 = A123E
Dmem[12] = 12345678

RESULTS EXPECTED

reg1 = 00061F31
reg4 = FFFFF5FE
reg7 = 12345678
reg8 = 00001238
reg10 = 0000162A

FULL CODE (contributed by everyone)

```
risc_processor.v x testbench.v x
[Search] [Save] [Undo] [Redo] [Cut] [Copy] [Paste] [Comment] [Lightbulb]

1 // Top module
2 module risc_processor (
3     input clk,
4     input reset,
5     output reg [31:0] pc,
6     output wire [31:0] r1, r4, r7, r8, r10,
7     output reg alusrc_, mr_, mw_, mreg_, enr_, ifidwrite_, pcwrite_, st_, enim_,
8     output reg [1:0] fa_, fb_, aluop_
9 );
10 // Pipeline register contents
11 reg [31:0] if_id_instruction, if_id_pc;
12 reg [31:0] id_ex_rd1, id_ex_rd2, id_ex_imm;
13 reg [3:0] id_ex_wn, id_ex_rn1, id_ex_rn2;
14 reg id_ex_alusrc, id_ex_mr, id_ex_mw, id_ex_mreg, id_ex_enr;
15 reg [1:0] id_ex_aluop;
16 reg [31:0] ex_mem_alu_result, ex_mem_rd2;
17 reg [3:0] ex_mem_wn;
18 reg ex_mem_mr, ex_mem_mw, ex_mem_mreg, ex_mem_enr;
19 reg [31:0] mem_wb_mem_data, mem_wb_alu_result;
20 reg [3:0] mem_wb_wn;
21 reg mem_wb_mreg, mem_wb_enr;
22
23 // Control signals
24 wire alusrc_, mr_, mw_, mreg_, enr_, ifidwrite_, pcwrite_, st_, enim_;
25 wire [1:0] aluop;
26 wire [1:0] fa_, fb_;
27 reg [31:0] pc_next;
28
29 // Update control signals outputs
30 always @(*) begin
31     if (reset) begin
32         alusrc_ <= 0;
33         mr_ <= 0;
34         mw_ <= 0;
35         mreg_ <= 0;
36         enr_ <= 0;
37         ifidwrite_ <= 0;
38         pcwrite_ <= 0;
39         st_ <= 0;
40         enim_ <= 0;
41         fa_ <= 0;
42         fb_ <= 0;
43         aluop_ <= 0;
44     end else begin
45         alusrc_ <= alusrc;
46         mr_ <= mr;
47         mw_ <= mw;
48         mreg_ <= mreg;
49         enr_ <= enr;
50         ifidwrite_ <= ifidwrite;
51         pcwrite_ <= pcwrite;
52         st <= st;
```

```

52         st_ <= st;
53         enim_ <= enim;
54         fa_ <= fa;
55         fb_ <= fb;
56         aluop_ <= aluop;
57     end
58 end
59
60 // PC update
61 always @(posedge clk or posedge reset) begin
62     if (reset) begin
63         pc <= 0;
64         pc_next <= 4;
65     end else if (pcwrite) begin
66         pc <= pc_next;
67         pc_next <= pc_next + 4;
68     end
69 end
70
71 // Instruction Fetch Stage
72 wire [31:0] instruction;
73 instruction_memory imem (
74     .addr(pc),
75     .instruction(instruction)
76 );
77
78 // IF/ID Pipeline Register
79 always @(posedge clk or posedge reset) begin
80     if (reset) begin
81         if_id_instruction <= 0;
82         if_id_pc <= 0;
83     end else if (ifidwrite) begin
84         if_id_instruction <= instruction;
85         if_id_pc <= pc;
86     end
87 end
88
89 // Instruction Decode Stage
90 wire [3:0] opcode = if_id_instruction[31:28];
91 wire [3:0] wn = if_id_instruction[27:24];
92 wire [3:0] rn1 = if_id_instruction[23:20];
93 wire [3:0] rn2 = if_id_instruction[19:16];
94 wire [15:0] imm = if_id_instruction[15:0];
95 wire [31:0] rd1, rd2;
96
97 // Control Unit
98 assign alusrc = (opcode == 4'b0011 || opcode == 4'b1111);
99 assign mr = (opcode == 4'b0011);
100 assign mw = 1'b0;
101 assign mreg = (opcode == 4'b0011);
102 assign enrw = (opcode == 4'b0000 || opcode == 4'b0001 || opcode == 4'b0011 || opcode == 4'b0111 || opcode == 4'b1111);
103 assign enim = 1'b1;

```

```

102 assign enrw = (opcode == 4'b0000 || opcode == 4'b0001 || opcode == 4'b0011 || opcode == 4'b0111 || opcode == 4'b1111);
103 assign enim = 1'b1;
104 assign aluop = (opcode == 4'b0000) ? 2'b01 : // SUB
105             (opcode == 4'b0001) ? 2'b11 : // NAND
106             (opcode == 4'b0111) ? 2'b10 : // AND
107             2'b00; // ADD for LW, ADDI
108
109 // Register File
110 register_file regfile (
111     .clk(clk),
112     .enrw(mem_wb_enrw),
113     .rn1(rn1),
114     .rn2(rn2),
115     .wn(mem_wb_wn),
116     .wd(mem_wb_mreg ? mem_wb_mem_data : mem_wb_alu_result),
117     .rd1(rd1),
118     .rd2(rd2),
119     .r1(r1),
120     .r4(r4),
121     .r7(r7),
122     .r8(r8),
123     .r10(r10)
124 );
125
126 // ID/EX Pipeline Register
127 always @(posedge clk or posedge reset) begin
128     if (reset) begin
129         id_ex_rd1 <= 0;
130         id_ex_rd2 <= 0;
131         id_ex_imm <= 0;
132         id_ex_wn <= 0;
133         id_ex_rn1 <= 0;
134         id_ex_rn2 <= 0;
135         id_ex_alusrc <= 0;
136         id_ex_aluop <= 0;
137         id_ex_mr <= 0;
138         id_ex_mw <= 0;
139         id_ex_mreg <= 0;
140         id_ex_enrw <= 0;
141     end else if (st) begin
142         // Stall - insert NOP
143         id_ex_alusrc <= 0;
144         id_ex_aluop <= 0;
145         id_ex_mr <= 0;
146         id_ex_mw <= 0;
147         id_ex_mreg <= 0;
148         id_ex_enrw <= 0;
149         id_ex_rd1 <= rd1;
150         id_ex_rd2 <= rd2;
151         id_ex_imm <= {{16{imm[15]}}, imm};
152         id_ex_wn <= wn;
153         id_ex_rn1 <= rn1;

```

```

152         id_ex_wn <= wn;
153         id_ex_rn1 <= rn1;
154         id_ex_rn2 <= rn2;
155     end else begin
156         id_ex_rd1 <= rd1;
157         id_ex_rd2 <= rd2;
158         id_ex_imm <= {{16{imm[15]}}}, imm};
159         id_ex_wn <= wn;
160         id_ex_rn1 <= rn1;
161         id_ex_rn2 <= rn2;
162         id_ex_alusrc <= alusrc;
163         id_ex_aluop <= aluop;
164         id_ex_mr <= mr;
165         id_ex_mw <= mw;
166         id_ex_mreg <= mreg;
167         id_ex_enrw <= enrw;
168     end
169 end

170
171 // Execute Stage
172 wire [31:0] alu_a = (fa == 2'b10) ? ex_mem_alu_result :
173                     (fa == 2'b01) ? (mem_wb_mreg ? mem_wb_mem_data : mem_wb_alu_result) :
174                     id_ex_rd1;
175
176 wire [31:0] alu_b = id_ex_alusrc ? id_ex_imm :
177                     ((fb == 2'b10) ? ex_mem_alu_result :
178                     (fb == 2'b01) ? (mem_wb_mreg ? mem_wb_mem_data : mem_wb_alu_result) :
179                     id_ex_rd2);
180
181 wire [31:0] alu_result;
182 alu alu_unit (
183     .op(id_ex_aluop),
184     .a(alu_a),
185     .b(alu_b),
186     .result(alu_result)
187 );
188
189 // EX/MEM Pipeline Register
190 always @(posedge clk or posedge reset) begin
191     if (reset) begin
192         ex_mem_alu_result <= 0;
193         ex_mem_rd2 <= 0;
194         ex_mem_wn <= 0;
195         ex_mem_mr <= 0;
196         ex_mem_mw <= 0;
197         ex_mem_mreg <= 0;
198         ex_mem_enrw <= 0;
199     end else begin
200         ex_mem_alu_result <= alu_result;
201         ex_mem_rd2 <= id_ex_rd2;
202         ex_mem_wn <= id_ex_wn;
203         ex_mem_mr <= id_ex_mr;

```



```

203         ex_mem_mr <= id_ex_mr;
204         ex_mem_mw <= id_ex_mw;
205         ex_mem_mreg <= id_ex_mreg;
206         ex_mem_enrw <= id_ex_enrw;
207     end
208 end
209
210 // Memory Stage
211 wire [31:0] mem_data;
212 data_memory dmem (
213     .clk(clk),
214     .mr(ex_mem_mr),
215     .mw(ex_mem_mw),
216     .addr(ex_mem_alu_result),
217     .data_in(ex_mem_rd2),
218     .data_out(mem_data)
219 );
220
221 // MEM/WB Pipeline Register
222 always @(posedge clk or posedge reset) begin
223     if (reset) begin
224         mem_wb_mem_data <= 0;
225         mem_wb_alu_result <= 0;
226         mem_wb_wn <= 0;
227         mem_wb_mreg <= 0;
228         mem_wb_enrw <= 0;
229     end else begin
230         mem_wb_mem_data <= mem_data;
231         mem_wb_alu_result <= ex_mem_alu_result;
232         mem_wb_wn <= ex_mem_wn;
233         mem_wb_mreg <= ex_mem_mreg;
234         mem_wb_enrw <= ex_mem_enrw;
235     end
236 end
237
238 // Hazard Detection Unit
239 hazard_detection hazard_unit (
240     .id_ex_memread(id_ex_mr),
241     .id_ex_wn(id_ex_wn),
242     .if_id_rn1(rn1),
243     .if_id_rn2(rn2),
244     .ifidwrite(ifidwrite),
245     .pcwrite(pcwrite),
246     .st(st)
247 );
248
249 // Forwarding Unit
250 forwarding_unit forward_unit (
251     .ex_mem_wn(ex_mem_wn),
252     .mem_wb_wn(mem_wb_wn),
253     .ex_mem_regwrite(ex_mem_enrw),
254     .mem_wb_readwrite(mem_wb_enrw).

```

```

254         .mem_wb_regwrite(mem_wb_enrw),
255         .id_ex_rn1(id_ex_rn1),
256         .id_ex_rn2(id_ex_rn2),
257         .fa(fa),
258         .fb(fb)
259     );
260 endmodule
261
262 // Instruction Memory Module
263 module instruction_memory (
264     input [31:0] addr,
265     output reg [31:0] instruction
266 );
267     reg [31:0] imem [0:7];
268
269     initial begin
270         imem[0] = 32'h01230000; // SUB r1, r2, r3
271         imem[1] = 32'h14150000; // NAND r4, r1, r5
272         imem[2] = 32'h37600003; // LW r7, 3(r6)
273         imem[3] = 32'h78790000; // AND r8, r7, r9
274         imem[4] = 32'hFA8003F2; // ADDI r10, r8, 1010
275         imem[5] = 0;
276         imem[6] = 0;
277         imem[7] = 0;
278     end
279
280     always @(*) begin
281         instruction = imem[addr[4:2]];
282     end
283 endmodule
284
285 // Register File Module
286 module register_file (
287     input clk,
288     input enrw,
289     input [3:0] rn1, rn2, wn,
290     input [31:0] wd,
291     output reg [31:0] rd1, rd2, r1, r4, r7, r8, r10
292 );
293     reg [31:0] registers [0:15];
294
295     initial begin
296         registers[0] = 32'h0;
297         registers[1] = 32'h0;
298         registers[2] = 32'h82344;
299         registers[3] = 32'h20413;
300         registers[4] = 32'h0;
301         registers[5] = 32'h16ACD;
302         registers[6] = 32'h9;
303         registers[7] = 32'h0;
304         registers[8] = 32'h0;
305         registers[9] = 32'hA123E;

```

```

306         registers[10] = 32'h0;
307         registers[11] = 32'h0;
308         registers[12] = 32'h0;
309         registers[13] = 32'h0;
310         registers[14] = 32'h0;
311         registers[15] = 32'h0;
312     end
313
314     always @(negedge clk)
315     begin
316         rd1 = registers[rn1];
317         rd2 = registers[rn2];
318     end
319     always @(*)
320     begin
321         r1 = registers[1];
322         r4 = registers[4];
323         r7 = registers[7];
324         r8 = registers[8];
325         r10 = registers[10];
326     end
327     always @(posedge clk) begin
328         if (enrw && wn != 0)
329             registers[wn] = wd;
330     end
331 endmodule
332
333 // ALU Module
334 module alu (
335     input [1:0] op,
336     input [31:0] a, b,
337     output reg [31:0] result
338 );
339     always @(*) begin
340         case (op)
341             2'b00: result = a + b; // ADD
342             2'b01: result = a - b; // SUB
343             2'b10: result = a & b; // AND
344             2'b11: result = ~(a & b); // NAND
345             default: result = 0;
346         endcase
347     end
348 endmodule
349
350 // Data Memory Module
351 module data_memory (
352     input clk,
353     input mr, mw,
354     input [31:0] addr,
355     input [31:0] data_in,
356     output reg [31:0] data_out
357 );

```

```

357 );
358     reg [31:0] dmem [0:15];
359
360     initial begin
361         dmem[12] = 32'h12345678; // Address 12
362     end
363
364     always @(*) begin
365         if (mr)
366             data_out = dmem[addr];
367         else
368             data_out = 32'h0;
369     end
370
371     // Write on positive edge
372     always @(posedge clk) begin
373         if (mw)
374             dmem[addr] <= data_in;
375     end
376 endmodule
377
378 // Hazard Detection Module
379 module hazard_detection (
380     input id_ex_memread,
381     input [3:0] id_ex_wn,
382     input [3:0] if_id_rn1, if_id_rn2,
383     output reg ifidwrite, pcwrite, st
384 );
385     always @(*) begin
386         if (id_ex_memread && id_ex_wn != 0 &&
387             (id_ex_wn == if_id_rn1 || id_ex_wn == if_id_rn2)) begin
388             ifidwrite = 0;
389             pcwrite = 0;
390             st = 1;
391         end else begin
392             ifidwrite = 1;
393             pcwrite = 1;
394             st = 0;
395         end
396     end
397 endmodule
398
399 // Forwarding Unit Module
400 module forwarding_unit (
401     input [3:0] ex_mem_wn, mem_wb_wn,
402     input ex_mem_regwrite, mem_wb_regwrite,
403     input [3:0] id_ex_rn1, id_ex_rn2,
404     output reg [1:0] fa, fb
405 );
406     always @(*) begin
407         // Forward A
408         if (ex_mem_regwrite && ex_mem_wn != 0 && ex_mem_wn == id_ex_rn1)

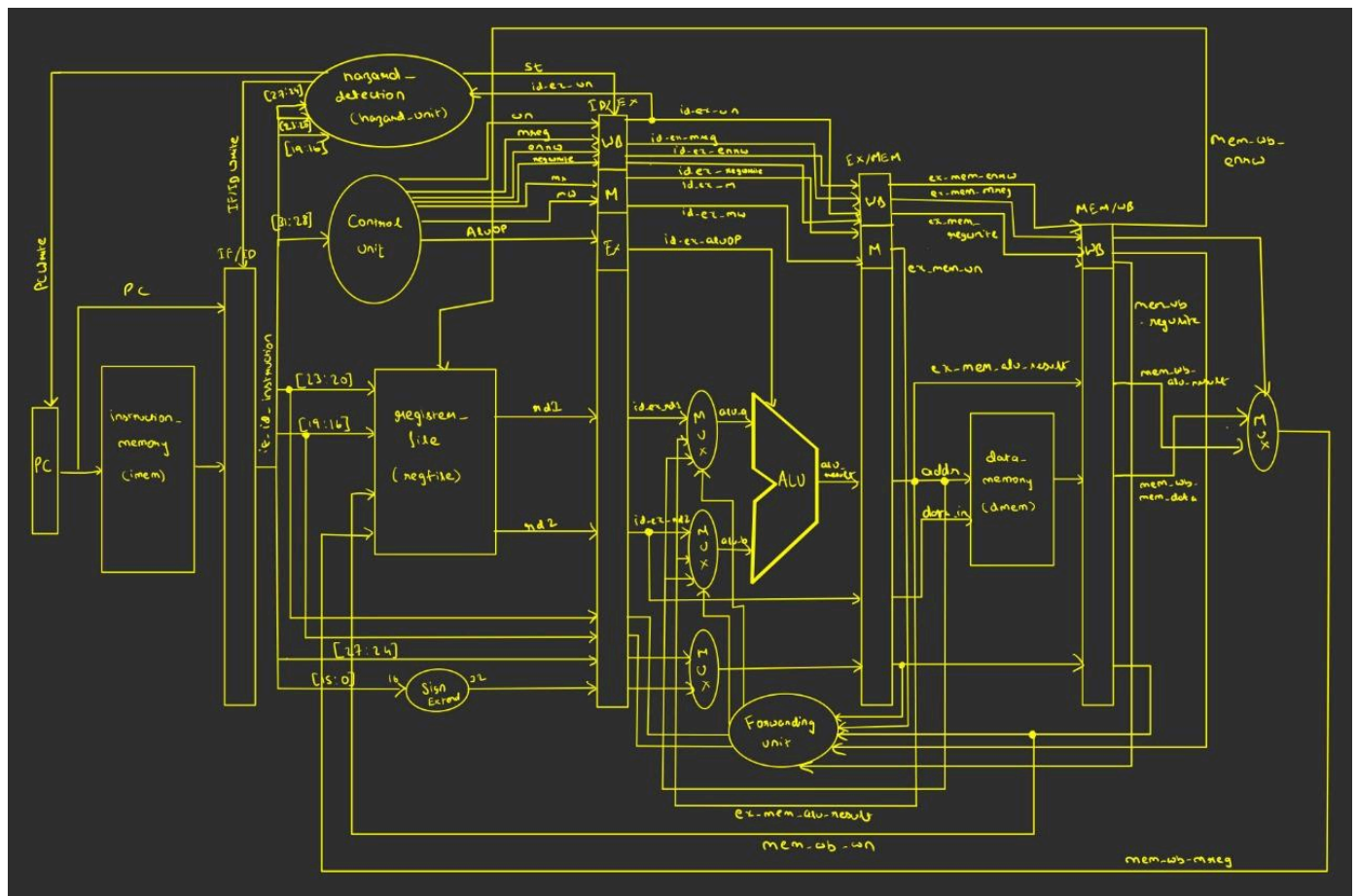
```

```

406 always @(*) begin
407     // Forward A
408     if (ex_mem_regwrite && ex_mem_wn != 0 && ex_mem_wn == id_ex_rn1)
409         fa = 2'b10;
410     else if (mem_wb_regwrite && mem_wb_wn != 0 && mem_wb_wn == id_ex_rn1)
411         fa = 2'b01;
412     else
413         fa = 2'b00;
414
415     // Forward B
416     if (ex_mem_regwrite && ex_mem_wn != 0 && ex_mem_wn == id_ex_rn2)
417         fb = 2'b10;
418     else if (mem_wb_regwrite && mem_wb_wn != 0 && mem_wb_wn == id_ex_rn2)
419         fb = 2'b01;
420     else
421         fb = 2'b00;
422 end
423 endmodule

```

PROCESSOR ARCHITECTURE (by VARUN PRAKASH)



MODULES(by HARSHIT VERMA)

- INSTRUCTION MEMORY MODULE
- REGFILE MODULE
- ALU MODULE
- HAZARD DETECTION MODULE
- DATA FORWARDING MODULE
- TOP MODULE

Pipeline registers are implemented as reg values in the top module itself.

Control unit has been implemented as a set of nets defined in the main module, which change value as soon as instruction in the ID/EX register is updated.

PC adder is again implemented in the top module directly.

BLOCKS EXPLAINED(by HANUSH NAG ABHIRAM M)

1. Program Counter (PC) Block (Inside top module)

Function:

- Holds the address of the current instruction
- Increments by 4 each clock cycle if pcwrite is high

Inputs:

- clk: system clock
- reset: synchronous reset
- pcwrite: control signal to enable PC update

Outputs:

- pc: 32-bit program counter value
- pc_next: internal signal for next instruction address

It is synchronous as we only want the PC to change at every clock cycle and not in the middle of one.

2. Instruction Memory (IMEM)

Function:

- Stores and outputs 32-bit instructions based on the PC value
- Acts as ROM for program code

Inputs:

- addr [31:0]: instruction address (from PC)

Outputs:

- instruction [31:0]: instruction fetched from memory.

There is a control signal for controlling the reading of instruction memory, but that is always set to 1 so it doesn't make any difference. This block is not synchronous as the IF/ID register writing is synchronous, so this doesn't need to be.

3. IF/ID Pipeline Register (Inside top module)**Function:**

- Latches the instruction and PC from the fetch stage to the decode stage

Signals Stored:

- if_id_instruction [31:0]: instruction->32 bits
- if_id_pc [31:0]: current PC->32 bits

Write Condition:

- Only written if ifidwrite is high

This only updates at every positive clock edge or at positive reset edge ensuring that different instructions don't overlap in this stage. This is also why the IMEM block doesn't need to be synchronous.

Size: 64 bits

4. Register File

Function:

- Provides two asynchronous read ports and one synchronous write port for 32-bit registers
- 16 general-purpose registers (32-bit wide)

Inputs:

- clk
- enrw: write enable
- rn1, rn2 [3:0]: read register addresses
- wn [3:0]: write register address
- wd [31:0]: data to be written (from WB stage)

Outputs:

- rd1, rd2 [31:0]: register read data
- Monitored outputs: r1, r4, r7, r8, r10

This block is synchronous. At the positive edge, the data is written into the regfile and at the negative edge it is read, to prevent data hazards. The outputs in testbench are updated by an always block asynchronously.

5. Control Unit (Inside top module)

Function:

- Generates control signals based on instruction opcode

Inputs:

- opcode [3:0]: from instruction bits 31–28

Outputs:

- alusrc, mr, mw, mreg, enrw, aluop [1:0], etc.
- These control the behavior of subsequent pipeline stages

These are defined as wires and not as a separate functional unit. We don't need to make this synchronous as the pipeline register ID/EX only updates at the posedge of clk so it does not matter, even if these signals are updated in the middle.

6. ID/EX Pipeline Register (Inside top module)

Function:

- Transfers decoded operands and control signals from ID to EX stage

Signals Stored:

- id_ex_rd1, id_ex_rd2 [31:0]: read register values -> 64 bits
- id_ex_imm [31:0]: sign-extended immediate -> 32 bits
- id_ex_wn, rn1, rn2 [3:0]: dest and source register indices -> 12 bits
- **Control signals:** id_ex_alusrc, id_ex_aluop [1:0], id_ex_mr, id_ex_mw, id_ex_mreg, id_ex_enrw -> 7 bits

Simple implementation with the reg variables defined in top module, which only update synchronously at posedge of clock.

Size: 115 bits

7. Forwarding Unit

Function:

- Detects data hazards and selects ALU input operands from previous stages to avoid stalls

Inputs:

- Register indices and write enable signals from EX/MEM and MEM/WB stages
- id_ex_rn1, id_ex_rn2: current source register indices

Outputs:

- fa, fb [1:0]: control signals for muxes selecting ALU inputs

This does not need to be synchronous as the update of control signals will only happen when the pipeline register ID/EX or EX/MEM or MEM/WBV values get updated, which already happens synchronously.

8. ALU Block

Function:

- Performs arithmetic and logical operations as determined by aluop

Inputs:

- a, b [31:0]: operands
- op [1:0]: ALU control (from control unit)

Outputs:

- result [31:0]: ALU output (used in EX/MEM register)

9. EX/MEM Pipeline Register (Inside top module)

Function:

- Transfers ALU result and relevant control/data to memory stage

Signals Stored:

- ex_mem_alu_result [31:0]: ALU result -> 32 bits
- ex_mem_rd2 [31:0]: second operand for memory write -> 32 bits
- ex_mem_wn [3:0]: destination register index -> 4 bits
- ex_mem_mr, ex_mem_mw, ex_mem_mreg, ex_mem_enrw: control signals -> 4 bits

Size: 72 bits

10. Data Memory (DMEM)

Function:

- Simulates RAM, supports asynchronous read and synchronous write

Inputs:

- clk, mr, mw
- addr [31:0]: memory address
- data_in [31:0]: data for write

Outputs:

- data_out [31:0]: data for read (used in MEM/WB register)

Reading happens asynchronously and writing happens on the positive edge of the clock. No mem write operations are performed here so this is redundant.

11. MEM/WB Pipeline Register (Inside top module)

Function:

- Holds final values to be written back to the register file

Signals Stored:

- mem_wb_mem_data [31:0]: data read from memory -> 32 bits
- mem_wb_alu_result [31:0]: ALU result -> 32 bits
- mem_wb_wn [3:0]: destination register -> 4 bits
- mem_wb_mreg, mem_wb_enrw: control for choosing source and write enabling -> 2 bits

Size: 70 bits

12. Hazard Detection Unit

Function:

- Detects load-use hazards that require pipeline stall

Inputs:

- id_ex_memread: indicates current instruction in EX is a LW
- id_ex_wn: write register of that instruction
- if_id_rn1, if_id_rn2: source registers of next instruction

Outputs:

- ifidwrite, pcwrite: to freeze IF/ID and PC
- st: to insert NOP in EX stage by making control signals 0

This block is not synchronous as again, it will only give output on change of pipeline register, which happens synchronously.

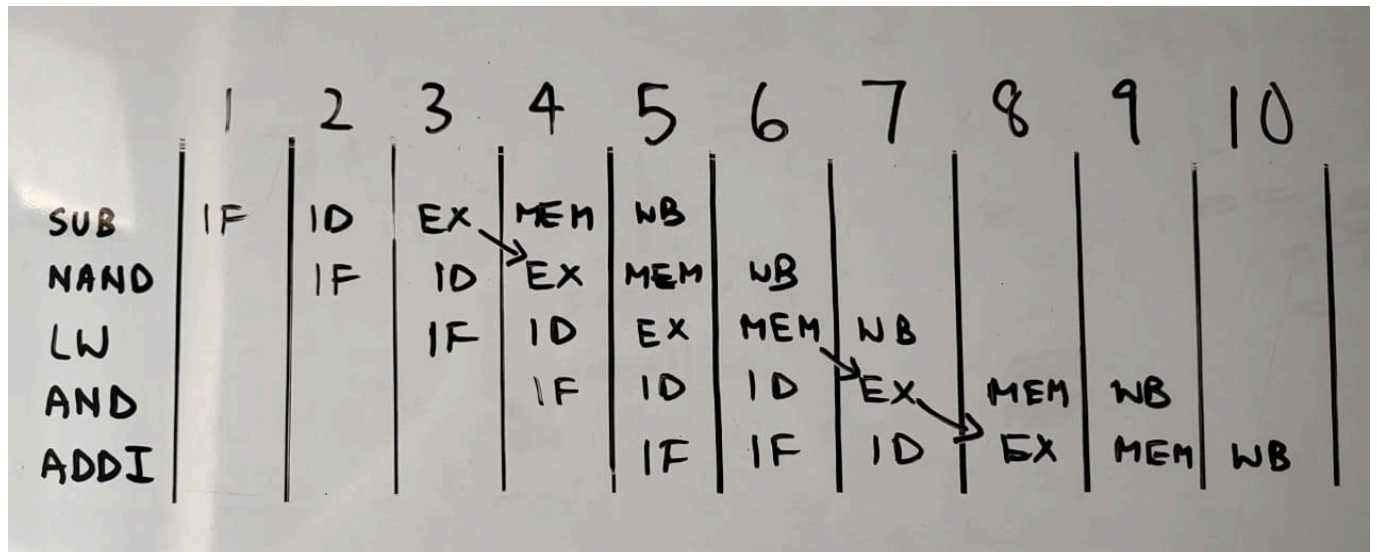
TESTBENCH(contributed by everyone)

```
risc_processor.v x testbench.v x
Q | | | | | | | | | |
1 // Testbench Module
2 module testbench;
3     reg clk = 0;
4     reg reset = 1;
5     wire [31:0] pc, r1, r4, r7, r8, r10;
6     wire alusrc_, mr_, mw_, mreg_, enrw_, ifidwrite_, pcwrite_, st_, enim_;
7     wire [1:0] fa_, fb_, aluop_;
8
9     risc_processor dut (
10         .clk(clk),
11         .reset(reset),
12         .pc(pc),
13         .alusrc_(alusrc_),
14         .mr_(mr_),
15         .mw_(mw_),
16         .mreg_(mreg_),
17         .enrw_(enrw_),
18         .ifidwrite_(ifidwrite_),
19         .pcwrite_(pcwrite_),
20         .st_(st_),
21         .enim_(enim_),
22         .fa_(fa_),
23         .fb_(fb_),
24         .aluop_(aluop_),
25         .r1(r1),
26         .r4(r4),
27         .r7(r7),
28         .r8(r8),
29         .r10(r10)
30     );
31
32     always #5 clk = ~clk;
33     initial
34     begin
35         #10;
36         reset = 0;
37         #110;
38         $finish;
39     end
40 endmodule
41
```

SIMULATION RESULT(contributed by everyone)



TIMELINE OF EXECUTION(by DEBANGSHU PATNAIK)



CYCLE BY CYCLE EXPLANATION(by DEBANGSHU PATNAIK and HARSHIT VERMA)

CYCLE 1:

Simple IF stage for instruction 1. Parallel to it for a small time, reset is set to 1 so as to initialize all the values to 0 for the starting of simulation. The PC is set to 0, and the IF for instruction 1 gets executed after reset goes to 0. This causes the immediate change in the control signals as we see. The values in the if_id register variables are initialized for instruction 1.

CYCLE 2:

ID stage gets executed for instruction 1, and IF for instruction 4 gets executed with PC going to 4 with the posedge of clk. ID being executed means the updated if_id register values now affect the control signals. Here there is no change in control signals as such, but this is what happens. Then the register values are read based on the if_id_rd1 and if_id_rd2 values and along with the relevant control signals and the 16 bit immediate value is passed to the ID/EX register variables.

Control signals(instruction 1 ID):

- 1) **enrw_** and **enim_** become and stay 1 throughout as all instructions perform reg write and because instruction memory is always read(we control the write into IF/ID register not the reading from instruction memory).
- 2) **ifidwrite_** and **pcwrite_** becomes 1, and **st** becomes 0, and remains the same unless a load instruction hazard is detected.
- 3) **fa_** and **fb_** are 0 until some forwarding needs to be done.
- 4) **mr_** and **mw_** are 0 as there is no memory involvement for instruction 1.
- 5) **mreg_** is 0 as the value of ALU is to be written into the register for this instruction.
- 6) **alusrc_** is 0 as it is R type only.
- 7) **aluop_** is 1 as it is a subtraction instruction.

CYCLE 3:

EX for instruction 1, ID for instruction 2 and IF for instruction 3.

In EX, the values from ID/EX register, i.e. here, the (id_ex_..) variables are used to do the relevant ALU operation. In case of a data forwarding the input of the ALU is controlled by the fa and fb signals.

Control signals(instruction 2 ID):

Only `aluop_` changes from 1 to 3 as we go from SUB to NAND.

CYCLE 4:

MEM for instruction 1, EX for instruction 2, ID for instruction 3 and IF for instruction 4. The first data forwarding happened here. The value from EX of instruction 1 in the previous cycle is passed for EX of instruction 2. Rest everything happens normally.

Control signals(instruction 3 ID):

- 1) `aluop_` changes from 3 to 0 as now it is an ADD operation to be performed.
- 2) `alusrc_` changes to 1 as now it is I type instruction.
- 3) `fa_` goes from 0 to 2 as now there is data forwarding between instructions 1 and 2.
- 4) `mr_` and `mreg_` are 1 now as for load both memory read needs to happen and data from memory needs to be written in the register file.

CYCLE 5:

WB for instruction 1, MEM for instruction 2, EX for instruction 3, ID for instruction 4 and IF for instruction 5. In WB, the value obtained in `mem_wb_aluresult` or `mem_wb_data` is written into `mem_wb_wn` register depending on the control signal of `mreg_`.

Control signals(instruction 4 ID):

- 1) `st_` changes to 1 as now the load instruction is in EX stage and the next instruction has a dependency on it. This causes a stall.
- 2) `fa_` changes to 0 as instruction 3 has no dependency on instruction 2.
- 3) `aluop_` changes to 2 for the AND instruction.
- 4) `alusrc_` goes back to 0 as AND is R type instruction.
- 5) `pcwrite_` and `idwrite_` go to 0 as `st_` goes to 1.
- 6) `mr_` and `mreg_` go back to 0 as no memory operations.

CYCLE 6:

As the stall due to load causes the ID and IF of instructions 4 and 5 to repeat.

Also as instruction 1 has completed, the computed value of instruction 1, 00061f31 is visible as the value in reg1. the instruction 2 completes its WB stage. Instruction 3 LOAD continues with its MEM stage.

Control Signals are of instruction 4 again as it is stalled.

But, **fa_ value changes to 2**, as there is indeed a dependency between the load instruction and the next instruction, causing the forwarding unit to be triggered even though we are introducing a stall.

Also obviously **st_** becomes 0 again now and **pcwrite_** and **ifidwrite_** return back to 1.

CYCLE 7:

Now instruction 3, LOAD has done MEM stage, so it forwards the value to instruction 4 AND, which is in its EX stage. Instruction 5 continues with the ID stage now. **The value of the instruction 2 result, ffff5fe is visible as reg4 as it has completed execution** . And instruction 3 is in WB stage.

Control signals(instruction 5 ID):

- 1) **alusrc_** goes to 1 as it is an I type instruction.
- 2) **aluop_** becomes 0 as it is an ADD instruction.
- 3) **fa_** becomes 1 now as there is a dependency between instructions 3 and 4 but now from MEM/WB and ID/EX unlike last cycle.

CYCLE 8:

We ignore the control signals generated from ID this cycle onwards as all instructions ID is done. Instruction 4 does its MEM stage and instruction 5 does its EX stage. Data is forwarded from instruction 4's EX calculated value in the last cycle to the EX of instruction 5. **The result of instruction 3, 12345678 is now visible as the reg7 output as the instruction has completed execution.**

Control signals: **fa_** becomes 2 as there is a data dependency between instruction 5 and 4.

CYCLE 9:

Instruction 4 completes its WB. Instruction 5 completes its MEM stage.

CYCLE 10:

Instruction 5 in WB stage and **the computed value of instruction 4, 00001238 is visible as reg8 output as it has completed execution.**

CYCLE 11:

The computed value of instruction 5, 0000162A is visible as reg10 output as it has completed execution. And that completes the execution of all 5 instructions.

EXTRA(by DEBANGSHU PATNAIK)

The events requiring explanation, i.e the ones involving data hazards and/or data forwarding are in cycles 4, 5&6 ,7 and 8.

In cycle 4, the data calculated from EX of the first instruction is forwarded to the second instruction, which is evident from:

- 1) The first instruction (SUB reg1, reg2, reg3) gives ALU result as 00061f31 in clock cycle 3, which hasn't been yet stored in the register file.
- 2) The second instruction (NAND reg4, reg1, reg5) has a dependency on the first instruction as it uses the reg1 value as input. This value of reg1 is forwarded from the first instruction to this. Because otherwise the output of ffff5fe is not possible with the actual stored value of reg1 in cycle 4, which is 0;
- 3) The fa control signal value is 2, signifying a dependency between the EX/MEM destination reg value and the ID/EX source reg 1 value.

In cycles 5&6, we see the hazard detection unit in action, as the third instruction, i.e. the load instruction(LW reg7, 3(reg6)) begins its EX stage, the hazard detection unit sets the st control signal to 1, which makes all the values of control signals 0 for that cycle, causing a stall for the following instructions 4 and 5. This is evident from the fact that the PC value remains the same for these two cycles and the output of the fourth and fifth instructions appear one clock cycle delayed than where they originally would have.

(Note: Control signals enim and enrw are still set to 1, and that is intended, because enim is always one, it is the writing into the IF/ID register which we turn off and enrw is needed, so that the instruction which is in the EX stage during stall can write back its value and is not interrupted from the stall)

In cycle 5 the fa value is 0, as there is no dependency.

In cycle 6, there is again a normal data dependency, if we don't take the stall into consideration. The EX/MEM dest reg value(ex_mem_wn variable), i.e. reg 7 from LW, and the source reg1 in ID/EX(id_ex_rn1), i.e. reg 7 from AND instruction, being the same, the fa control signal is set to 2.

Now the stall is very perfectly displayed by the fa control signal changing from 2 to 1 **in cycle 7**, as now the load instruction enters into MEM stage, so there is a dependency between the MEM/WB destreg(mem_wb_wn variable) value and ID/EX source reg1(id_ex_rn1), again the same reg7.

In cycle 8, again it is a normal data dependency between the fourth instruction, AND reg8, reg7, reg9, and the fifth instruction, ADDI reg10, reg8, 1010, because of the reg 8 value. This is again shown by the fa signal going to 2 from 1.

-----X-----